

Задание

на практическое занятие

по теме

«Расчёт параметров кэш-памяти»

Целями занятия являются:

- закрепление знаний об ассоциативном способе адресации памяти,
- получение представления о типах кэш-памяти,
- овладение навыками расчёта параметров кэш-памяти.

Внимание:

В русскоязычной литературе терминология, касающаяся описания структуры кэш-памяти, к сожалению, всё ещё не является устоявшейся и общепринятой. Так под понятием «строка кэш-памяти» в разных источниках могут подразумевать: а) просто блок данных, б) непосредственно связку «тэг + данные», а так же: в) наборы подобных «связок» в случае множественно-ассоциативного кэша. Путаница возникает из-за использования в оригинальных источниках термина **line** (он же **data block**) - «**cache line is the unit of data transfer between the cache and main memory**» и который обозначает непосредственно «блок данных, перемещаемый между основной и кэш-памятью», который и переводят иногда как «линия» или как блок данных, но чаще – как «строка» кэш-памяти. При этом, однако, в русскоязычной учебной литературе под строкой уже подразумевают непосредственно физическую «запись», формируемую в кэш-памяти, т.е. связку «тэг+блок данных», и которая в оригинальных источниках чаще всего обозначается как «**cache entry**».

Так же в случае варианта множественно-ассоциативного кэша, который в русскоязычных источниках называют «многоканальным», «многоходовым», «n-мерным» и т.п., под термином «набор» (set) в первоначальных источниках подразумевают несколько связок «тэг+данные», проверяемых одновременно при одном запросе на наличие искомых данных. При этом под термином **way** (который на русский переводят как «вход» или «канал») чаще всего подразумевают какой-то один из компараторов, сравнивающих тэг из текущей связки «тэг+данные» с тэгом искомого слова, извлекаемого из старших бит его адреса. Именно на данных компараторах и производится одновременная параллельная проверка (см. рис.3) наличия нужного слова в наборе строк кэша. К сожалению, из-за схожести перевода на русский язык понятий «entry» и «way», и которые в некоторых русскоязычных учебных пособиях переводят одним словом «вход», и возникает дополнительная путаница.

В соответствии с вышесказанным, во избежание неоднозначности трактования нижеприведённого задания, в рамках курса «Introduction to Digital Technologies», под **«строкой кэш-памяти»** мы будем подразумевать чуть более расширенное понятие чем просто «блок или пакет данных, перемещаемых из ОЗУ в кэш», а именно - связку **«тэг+блок данных»**, хранимые в кэш-памяти (т.е. по сути - cache entry), а под **«набором»** (set) – несколько подобных строк (две, четыре и т.п. – кратно степени двойки), параллельно посылаемых на проверку при поступлении запроса на наличие слова в кэше. Термином **«вход»** (реже - **«канал»**), в зависимости от контекста, будем обозначать либо отдельный компаратор либо некоторое множество строк кэш-памяти, которые могут поступить на проверку именно на данный компаратор - по отдельности и в разные моменты времени, естественно. Служебная информация (например: флаги, счётчики циклов с последнего чтения/записи строки и т.п.), хранимая для каждой строки в k-мерной и полностью ассоциативной кэш-памяти, в рамках текущего задания не рассматривается.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Быстродействие и цена микросхем памяти являются обратно-пропорциональными величинами. С целью минимизации стоимости вычислительных систем при сохранении их производительности на требуемом уровне используется иерархическая схема организации памяти. Помимо внутренней памяти центрального процессора (регистров) на кристалле ЦП располагается кэш-память первого L1, второго L2 и третьего уровня L3. Так же, уже выделяют в отдельное понятие «L4 – кэш», – некоторую часть кэша третьего уровня, физически доступную не только ЦП, но и GPU (- графического процессора) расположенного с ЦП на одном кристалле.

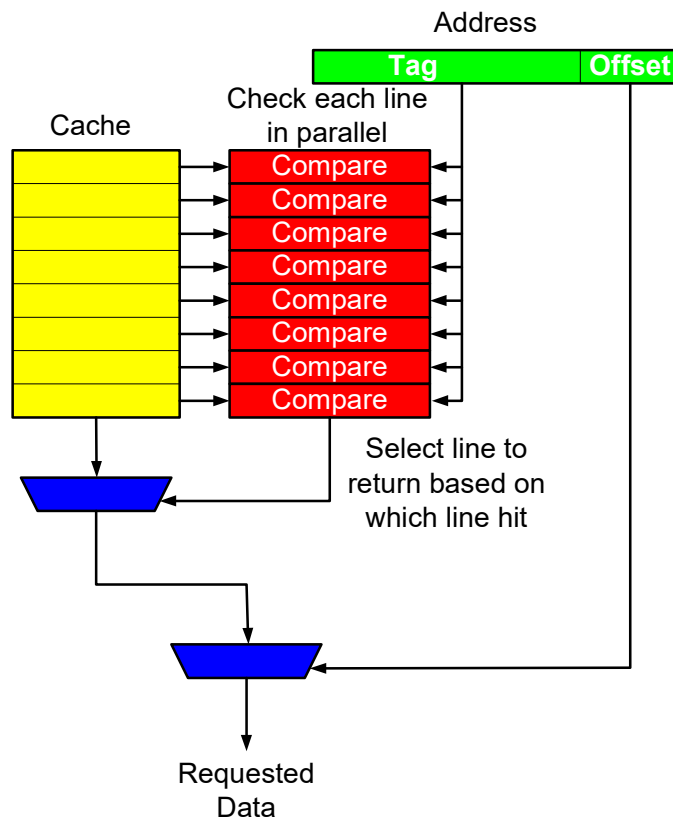


Рис. 1. Организация кэш-памяти с полностью ассоциативным отображением
По внутренней структуре кэш можно разделить на три типа:

- кэш с полностью ассоциативным отображением (рис. 1)
(fully-associative cache);
- кэш с прямым отображением (direct mapped cache);
- кэш с множественно-ассоциативным отображением (k-way associative cache).

В кэш-памяти с полностью ассоциативным отображением любой блок данных из основной памяти может быть продублирован в любом месте (т.е. в любой строке) кэш. Таким образом, кэш данного типа не может производить замещение хранимых в ней данных до тех пор, пока в наличие имеются свободные строки. Схема организации данного типа памяти приведена на рис. 1

В кэш-памяти с прямым отображением (direct mapped cache), каждому блоку памяти поставлена в соответствие отдельная позиция (строка) кэш-памяти. Таким образом, конкретный блок может храниться только в конкретной строке кэш-памяти. В случае, если требуется записать блок данных из основной памяти в конкретную строку кэш, то запись производится в любом случае – была данная строка занята или нет, не смотря на то, что соседние строки могут быть в это время свободными. Иными словами, замещение данных в кэш-памяти подобного типа и, соответственно, использование всей кэш-памяти производится не самым рациональным способом.

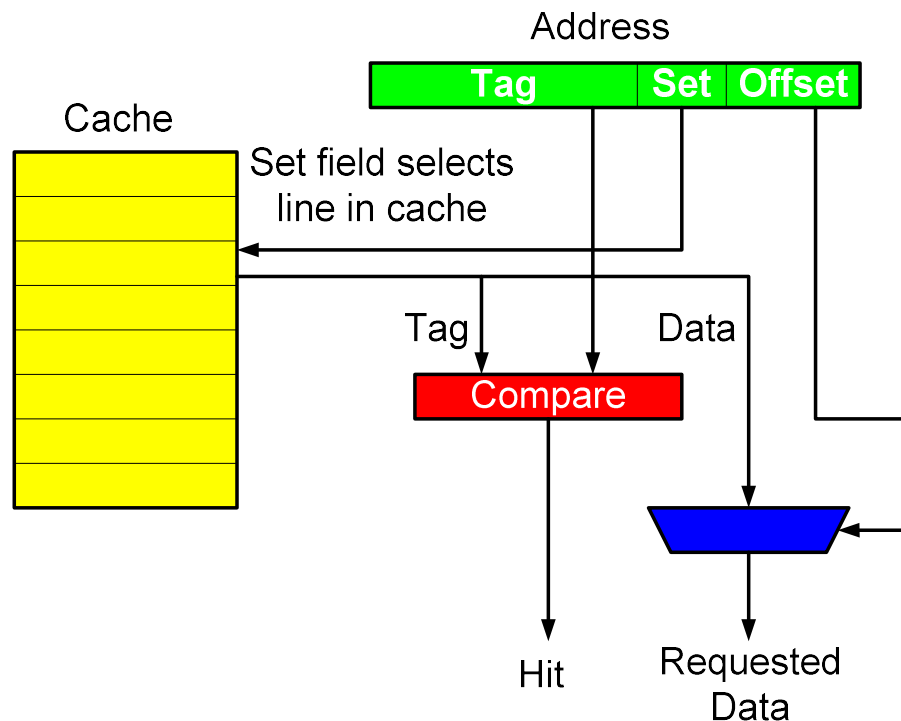


Рис. 2. Организация кэш-памяти с прямым отображением.

Для того, чтобы определить какой блок данных в какую строку кэш записывать в кэш-памяти с прямым отображением обычно используют несколько бит адреса для задания «отражения» (- от оригинального «mapping») блоков памяти на кэш. Например, биты 2 и 3 адреса (в зависимости от размера блока и строки) могут быть индексом, задающим номера строк кэш.

Схема организации данного типа памяти приведена на рис. 2.

Достоинства и недостатки обоих типов можно свести к следующим тезисам:

- Кэш с прямым отображением проще:
 - Занимает меньше места - меньше аппаратуры, малое количество бит требуется на хранение тэга - tag;
 - потенциально более быстрый;
 - но **б**ольшее количество промахов и замещений – меньший КПД.
- Кэш с полностью ассоциативным отображением:
 - меньше промахов и конфликтов – лучше производительность в итоге.
 - но требует отдельного компаратора для каждой строки.

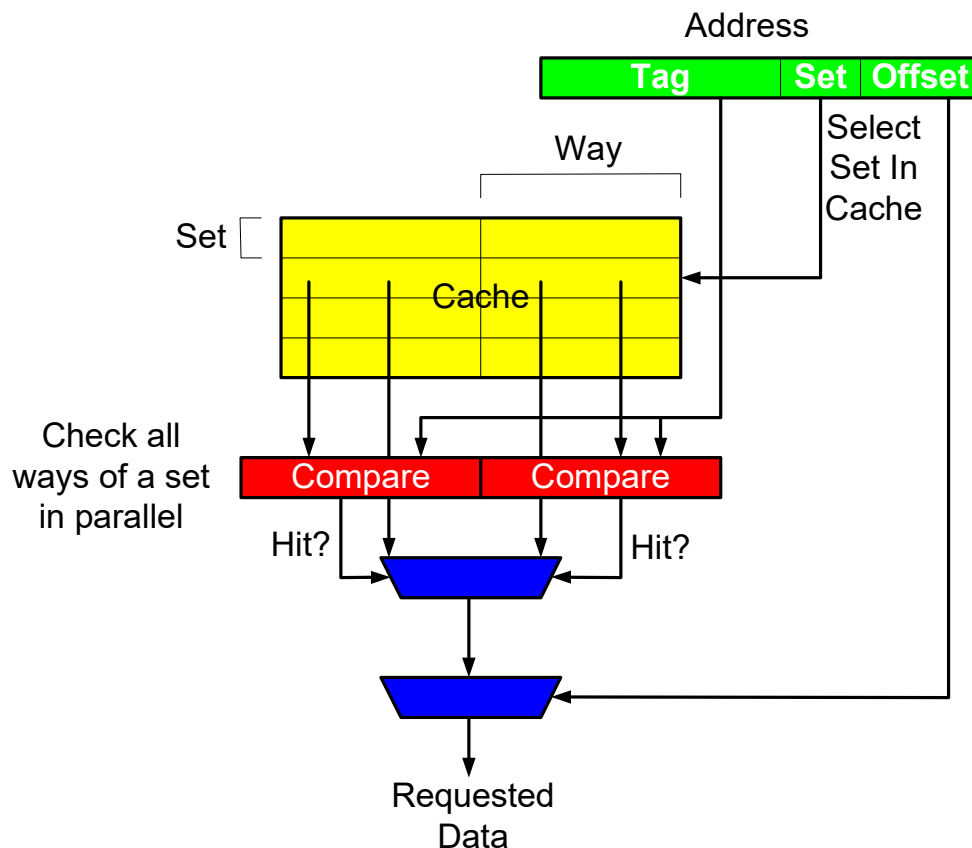


Рис. 3 Организация кэш-памяти с множественно-ассоциативным отображением (k=2)

Для совмещения преимуществ обоих способов появилась схема с множественно-ассоциативным отображением (k-way associative cache).

В "k-way set associative cache" индекс определяет «множество наборов по k строк кэш в каждом» (- set), где могут храниться данные, именно по этой причине в данном типе кэш-памяти этот параметр называется SET. В зависимости от значения параметра k, данная организация может вырождаться в вышерассмотренные случаи:

- при k=1 – полностью ассоциативное отображение (fully-associative cache).
- При k= количеству строк – прямое отображение (direct mapped)

2. Практическая ЧАСТЬ

Варианты задания представлены в виде строк в таблице. Пример:

№ комбинации	Объём ОЗУ (Гб)	ШД	Слов в строке	Делитель кэш	k
253	8	32	64	256	4

Необходимо вычислить разрядность полей (TAG, SET/INDEX, OFFSET) адреса слова ОЗУ, по которым происходит его поиск и (в случае нахождения, естественно) извлечение из кэш-памяти для трёх различных типов – полностью ассоциативного, множественно-ассоциативного (с заданным по варианту количеством входов – k) и прямого отображения.

- 2.1 Расчёт параметров кэш-памяти начинается прежде всего с определения размера самой кэш-памяти. В рамках задания настоящей практической работы размер кэш-памяти определяется путём деления заданного объёма ОЗУ (в данном случае 8 Гб) на «делитель» кэш-памяти – 256. Примечание: понятие делитель – весьма условное и на практике, как правило не встречается, оно использовано в данном случае только для удобства генерирования вариантов. Он обозначает лишь – «во сколько раз объём кэш-памяти меньше объёма ОЗУ».

Для текущего примера объём кэш-памяти будет равен: $8 * 2^{30} / 256 = 32\text{Mb}$

- 2.2 Следующим шагом определим количество бит, необходимых для адресации заданного объёма ОЗУ (т.е. фактически – разрядность шины адреса).

В примере: $ША = \log_2(8\text{GB}) = 33 \text{ бита}$

- 2.3 Самым простым параметром из задания, который мы можем определить – это смещение в строке кэш (OFFSET), которое будет одинаковым для всех типов кэш-памяти. В рамках данной работы мы делаем допущение, что адресация в ОЗУ (и следовательно в кэш-памяти) происходит ПОСЛОВНО, и соответственно количество бит смещения будет равным $\log_2(\text{количество слов с строке кэш})$.

Для текущего примера: $OFFSET = \log_2(64) = 6 \text{ бит}$.

Примечание: на самом деле, в разных реализациях кэш-памяти используется как побайтная, так и пословная адресация.

- 2.4 Остальные параметры (TAG и SET/INDEX) зависят от конкретного типа кэш-памяти.

Параметр TAG вычисляется на самом последнем шаге, а параметры SET/INDEX – по сути представляют собой одно и то же – «номер набора строк» и «номер одной строки» кэш, но для разных типов – k-мерного и прямого отображения, соответственно. В рамках данного ПЗ оба этих параметров будем обозначать одним названием – SET. Прим: так как разрядность шины данных дана в битах, а все вычисления мы производим в байтах, то для определения размера строки в байтах необходимо разделить параметр ШД на 8 бит и умножить на количество слов в строке. В текущем примере размер строки в байтах = $64 * 32 / 8 = 256 \text{ байт}$.

- 2.4.1 Для кэш-памяти прямого отображения вычисление параметра SET производится путём деления размера кэш на размер одной её строки и вычисление двоичного логарифма от получившегося количества строк.. Примечание: в данном случае за объём строки берётся только объём полезных данных, хранимых в ней. Биты тэга

не учитываются. Для текущего примера это будет равно: $SET_{k=1} =$

$$\log_2(32\text{Mb}/256) = 17$$

- 2.4.2 Для кэш k-мерной ассоциативности вычисление параметра SET производится не для каждой строки, а для набора из k строк, соответственно можно от полученного на предыдущем шаге значения отнять логарифм двоичный от k.

Для текущего примера это будет равно: $SET_{k=4} = 17 - \log_2(4) = 15$

- 2.4.3 Так как в случае полностью ассоциативной кэш-памяти любая строка может располагаться в любом её месте, то параметр SET для неё всегда равен 0 независимо от других параметров.

- 2.5 На последнем шаге вычисляется TAG – старшие биты адреса, которые хранятся в кэш-памяти вместе со строкой и по которым непосредственно и производится ассоциативный поиск. Соответственно для вычисления размера поля TAG необходимо отнять от количества бит полного адреса суммарное количество бит уже вычисленных полей – SET и OFFSET. Для рассматриваемого примера это будет:

- 2.5.1 В случае k=1 (прямого отображения): $TAG_{k=1} = 33 - 6 - 17 = 10$

- 2.5.2 В случае k=4 (k- мерной ассоциативности): $TAG_{k=4} = 33 - 6 - 15 = 12$

- 2.5.3 В случае k= кол-ву строк (полностью ассоциативный): $TAG_{k=} = 33 - 6 - 0 = 27$.

Таким образом полученные ответы можно записать в следующем виде:

k=1 (прямой доступ)			k=4 (множ.-ассоц.)			k=строк в кэше (полностью ассоциативный)		
TAG	SET	OFFSET	TAG	SET	OFFSET	TAG	SET	OFFSET
10	17	6	12	15	6	27	0	6

3. ХОД ВЫПОЛНЕНИЯ ЗАДАНИЯ:

1. Получить индивидуальное задание в соответствии с вариантом (см. прил. по номеру группы).
2. Произвести вычисления в соответствии с приведённой выше последовательностью шагов.
3. Сверить полученный ответ с ключами у преподавателя.